

Mini RFC: 0-RTT

April 21, 2022

Note: This is a simplified and annotated summary of the QUIC-related IETF RFCs. It's part of a course project at Tsinghua University and not suitable for any other purposes.

We reorder paragraphs in QUIC RFCs and omit many details to improve its readability for beginners.

WARNING: DO NOT distribute this document. Modification of extracts from IETF RFCs is not granted by IETF as original authors retain copyrights of RFCs. This document is for educational purposes only.

1 Overview

The 0-RTT feature in QUIC allows a client to send application data before the handshake is complete. This is made possible by reusing negotiated parameters from a previous connection. To enable this, 0-RTT depends on the client remembering critical parameters and providing the server with a TLS session ticket that allows the server to recover the same information.

This information includes parameters that determine TLS state, as governed by [RFC 8446], QUIC transport parameters, the chosen application protocol, and any information the application protocol might need. This information determines how 0-RTT packets and their contents are formed.

To ensure that the same information is available to both endpoints, all information used to establish 0-RTT comes from the same connection. Endpoints cannot selectively disregard information that might alter the sending or processing of 0-RTT.

[RFC 8446] sets a limit of seven days on the time between the original connection and any attempt to use 0-RTT. There are other constraints on 0-RTT usage, notably those caused by the potential exposure to replay attack.

1.1 Enabling 0-RTT

The TLS early_data extension in the NewSessionTicket message is defined to convey (in the max_early_data_size parameter) the amount of TLS 0-RTT data the server is willing to accept. QUIC does not use TLS early data. QUIC uses 0-RTT packets to carry early data. Accordingly, the max_early_data_size parameter is repurposed to hold a sentinel value 0xffffffff to indicate that

the server is willing to accept QUIC 0-RTT data. To indicate that the server does not accept 0-RTT data, the `early_data` extension is omitted from the `NewSessionTicket`. The amount of data that the client can send in QUIC 0-RTT is controlled by the `initial_max_data` transport parameter supplied by the server.

Servers MUST NOT send the `early_data` extension with a `max_early_data_size` field set to any value other than `0xffffffff`. A client MUST treat receipt of a `NewSessionTicket` that contains an `early_data` extension with any other value as a connection error of type `PROTOCOL_VIOLATION`.

A client that wishes to send 0-RTT packets uses the `early_data` extension in the `ClientHello` message of a subsequent handshake; see Section 4.2.10 of [RFC 8446]. It then sends application data in 0-RTT packets.

If 0-RTT is possible, it is ready after the client sends a TLS `ClientHello` message or the server receives that message. After providing a QUIC client with the first handshake bytes, the TLS stack might signal the change to 0-RTT keys. On the server, after receiving handshake bytes that contain a `ClientHello` message, a TLS server might signal that 0-RTT keys are available.

A client that attempts 0-RTT might also provide an address validation token if the server has sent a `NEW_TOKEN` frame.

See *mini RFC: Address Validation*

1.2 Accepting and Rejecting 0-RTT

A server accepts 0-RTT by sending an `early_data` extension in the `EncryptedExtensions`; see Section 4.2.10 of [RFC 8446]. The server then processes and acknowledges the 0-RTT packets that it receives.

A server rejects 0-RTT by sending the `EncryptedExtensions` without an `early_data` extension. A server will always reject 0-RTT if it sends a TLS `HelloRetryRequest`. When rejecting 0-RTT, a server MUST NOT process any 0-RTT packets, even if it could. When 0-RTT was rejected, a client SHOULD treat receipt of an acknowledgment for a 0-RTT packet as a connection error of type `PROTOCOL_VIOLATION`, if it is able to detect the condition.

When 0-RTT is rejected, all connection characteristics that the client assumed might be incorrect. This includes the choice of application protocol, transport parameters, and any application configuration. The client therefore MUST reset the state of all streams, including application state bound to those streams.

A client MAY reattempt 0-RTT if it receives a `Retry` or `Version Negotiation` packet. These packets do not signify rejection of 0-RTT.

1.3 Validating 0-RTT Configuration

When a server receives a `ClientHello` with the `early_data` extension, it has to decide whether to accept or reject 0-RTT data from the client. Some of this decision is made by the TLS stack (e.g., checking that the cipher suite being resumed was included in the `ClientHello`; see Section 4.2.10 of [RFC 8446]). Even when the TLS stack has no reason to reject 0-RTT data, the QUIC stack

or the application protocol using QUIC might reject 0-RTT data because the configuration of the transport or application associated with the resumed session is not compatible with the server's current configuration.

QUIC requires additional transport state to be associated with a 0-RTT session ticket. One common way to implement this is using stateless session tickets and storing this state in the session ticket. Application protocols that use QUIC might have similar requirements regarding associating or storing state. This associated state is used for deciding whether 0-RTT data must be rejected. For example, HTTP/3 settings [HTTP/3] determine how 0-RTT data from the client is interpreted. Other applications using QUIC could have different requirements for determining whether to accept or reject 0-RTT data.

2 Use of 0-RTT Keys

If 0-RTT keys are available, the lack of replay protection means that restrictions on their use are necessary to avoid replay attacks on the protocol.

Of the frames, the STREAM, RESET_STREAM, STOP_SENDING, and CONNECTION_CLOSE frames are potentially unsafe for use with 0-RTT as they carry application data. Application data that is received in 0-RTT could cause an application at the server to process the data multiple times rather than just once. Additional actions taken by a server as a result of processing replayed application data could have unwanted consequences. A client therefore **MUST NOT** use 0-RTT for application data unless specifically requested by the application that is in use.

An application protocol that uses QUIC **MUST** include a profile that defines acceptable use of 0-RTT; otherwise, 0-RTT can only be used to carry QUIC frames that do not carry application data.

Though replaying packets might result in additional connection attempts, the effect of processing replayed frames that do not carry application data is limited to changing the state of the affected connection. A TLS handshake cannot be successfully completed using replayed packets.

A client **MAY** wish to apply additional restrictions on what data it sends prior to the completion of the TLS handshake.

A client otherwise treats 0-RTT keys as equivalent to 1-RTT keys, except that it cannot send certain frames with 0-RTT keys.

A client that receives an indication that its 0-RTT data has been accepted by a server can send 0-RTT data until it receives all of the server's handshake messages. A client **SHOULD** stop sending 0-RTT data if it receives an indication that 0-RTT data has been rejected.

A server **MUST NOT** use 0-RTT keys to protect packets; it uses 1-RTT keys to protect acknowledgments of 0-RTT packets. A client **MUST NOT** attempt to decrypt 0-RTT packets it receives and instead **MUST** discard them.

Once a client has installed 1-RTT keys, it **MUST NOT** send any more 0-RTT packets.

If you have implemented the QUIC handshake, you will be familiar with the availability of crypto keys are installed at different phases of a TLS handshake.

0-RTT data can be acknowledged by the server as it receives it, but any packets containing acknowledgments of 0-RTT data cannot have packet protection removed by the client until the TLS handshake is complete. The 1-RTT keys necessary to remove packet protection cannot be derived until the client receives all server handshake messages.

3 Discarding 0-RTT Keys

0-RTT and 1-RTT packets share the same packet number space, and clients do not send 0-RTT packets after sending a 1-RTT packet.

Therefore, a client SHOULD discard 0-RTT keys as soon as it installs 1-RTT keys as they have no use after that moment.

Additionally, a server MAY discard 0-RTT keys as soon as it receives a 1-RTT packet. However, due to packet reordering, a 0-RTT packet could arrive after a 1-RTT packet. Servers MAY temporarily retain 0-RTT keys to allow decrypting reordered packets without requiring their contents to be retransmitted with 1-RTT keys. After receiving a 1-RTT packet, servers MUST discard 0-RTT keys within a short time; the RECOMMENDED time period is three times the Probe Timeout (PTO). A server MAY discard 0-RTT keys earlier if it determines that it has received all 0-RTT packets, which can be done by keeping track of missing packet numbers.

Related optional feature: *Probe Timeout*.

4 Transport Parameters for 0-RTT

Using 0-RTT depends on both client and server using protocol parameters that were negotiated from a previous connection. To enable 0-RTT, endpoints store the values of the server transport parameters with any session tickets it receives on the connection. Endpoints also store any information required by the application protocol or cryptographic handshake. The values of stored transport parameters are used when attempting 0-RTT using the session tickets.

Remembered transport parameters apply to the new connection until the handshake completes and the client starts sending 1-RTT packets. Once the handshake completes, the client uses the transport parameters established in the handshake. Not all transport parameters are remembered, as some do not apply to future connections or they have no effect on the use of 0-RTT.

A client MUST NOT use remembered values for the following parameters: `ack_delay_exponent`, `max_ack_delay`, `initial_source_connection_id`, `original_destination_connection_id`, `preferred_address`, `retry_source_connection_id`, and `stateless_reset_token`. The client MUST use the server's new values in the handshake instead; if the server does not provide new values, the default values are used.

A client that attempts to send 0-RTT data MUST remember all other transport parameters used by the server that it is able to process. The

All flow control related specifications are mandatory only if you implement optional features *Connection Flow Control* or *Stream Flow Control*.

server can remember these transport parameters or can store an integrity-protected copy of the values in the ticket and recover the information when accepting 0-RTT data. A server uses the transport parameters in determining whether to accept 0-RTT data.

If 0-RTT data is accepted by the server, the server **MUST NOT** reduce any limits or alter any values that might be violated by the client with its 0-RTT data. In particular, a server that accepts 0-RTT data **MUST NOT** set values for the following parameters that are smaller than the remembered values of the parameters.

- `active_connection_id_limit`
- `initial_max_data`
- `initial_max_stream_data_bidi_local`
- `initial_max_stream_data_bidi_remote`
- `initial_max_stream_data_uni`
- `initial_max_streams_bidi`
- `initial_max_streams_uni`

Omitting or setting a zero value for certain transport parameters can result in 0-RTT data being enabled but not usable. The applicable subset of transport parameters that permit the sending of application data **SHOULD** be set to non-zero values for 0-RTT. This includes `initial_max_data` and either (1) `initial_max_streams_bidi` and `initial_max_stream_data_bidi_remote` or (2) `initial_max_streams_uni` and `initial_max_stream_data_uni`.

A server might provide larger initial stream flow control limits for streams than the remembered values that a client applies when sending 0-RTT. Once the handshake completes, the client updates the flow control limits on all sending streams using the updated values of `initial_max_stream_data_bidi_remote` and `initial_max_stream_data_uni`.

A server **MAY** store and recover the previously sent values of the `max_idle_timeout`, `max_udp_payload_size`, and `disable_active_migration` parameters and reject 0-RTT if it selects smaller values. Lowering the values of these parameters while also accepting 0-RTT data could degrade the performance of the connection. Specifically, lowering the `max_udp_payload_size` could result in dropped packets, leading to worse performance compared to rejecting 0-RTT data outright.

A server **MUST** reject 0-RTT data if the restored values for transport parameters cannot be supported.

When sending frames in 0-RTT packets, a client **MUST** only use remembered transport parameters; importantly, it **MUST NOT** use updated values that it learns from the server's updated transport parameters or from frames received in 1-RTT packets. Updated values of transport parameters from the handshake apply only to 1-RTT packets. For instance, flow control limits from remembered transport parameters apply to all 0-RTT packets

even if those values are increased by the handshake or by frames sent in 1-RTT packets. A server MAY treat the use of updated transport parameters in 0-RTT as a connection error of type `PROTOCOL_VIOLATION`.

5 Packets & Frames

5.1 0-RTT Packet

A 0-RTT packet uses long headers with a type value of `0x01`, followed by the Length and Packet Number fields. The first byte contains the Reserved and Packet Number Length bits. A 0-RTT packet is used to carry “early” data from the client to the server as part of the first flight, prior to handshake completion. As part of the TLS handshake, the server can accept or reject this early data.

```
0-RTT Packet {  
    Header Form (1) = 1,  
    Fixed Bit (1) = 1,  
    Long Packet Type (2) = 1,  
    Reserved Bits (2),  
    Packet Number Length (2),  
    Version (32),  
    Destination Connection ID Length (8),  
    Destination Connection ID (0..160),  
    Source Connection ID Length (8),  
    Source Connection ID (0..160),  
    Length (i),  
    Packet Number (8..32),  
    Packet Payload (8..),  
}
```

Figure 1: 0-RTT Packet

Packet numbers for 0-RTT protected packets use the same space as 1-RTT protected packets.

After a client receives a Retry packet, 0-RTT packets are likely to have been lost or discarded by the server. A client SHOULD attempt to resend data in 0-RTT packets after it sends a new Initial packet. New packet numbers MUST be used for any new packets that are sent, reusing packet numbers could compromise packet protection.

A client only receives acknowledgments for its 0-RTT packets once the handshake is complete.

A client MUST NOT send 0-RTT packets once it starts processing 1-RTT packets from the server. This means that 0-RTT packets cannot contain any response to frames from 1-RTT packets. For instance, a client cannot send

an ACK frame in a 0-RTT packet, because that can only acknowledge a 1-RTT packet. An acknowledgment for a 1-RTT packet MUST be carried in a 1-RTT packet.

A server SHOULD treat a violation of remembered limits as a connection error of an appropriate type (for instance, a `FLOW_CONTROL_ERROR` for exceeding stream data limits).